

Lab Sheet 01

Supportive Document for VS code

Title: Create and Run ESP32 Project using Wokwi Simulator and platformIO extension in Visual Studio Code.

Part A: Setup Wokwi in Visual Studio Code

1. Install Required Extensions in VS Code
 - Open Visual Studio Code.
 - Go to Extensions.
 - Install the following:
 - Wokwi Simulator
 - PlatformIO
2. Activate Wokwi License
 - Press F1 to open Command Palette.
 - Type and select: Wokwi: Request a new License
 - Click Open when prompted.
 - It will open your browser. Log in to your Wokwi account.
 - Click GET YOUR LICENSE.
 - Return to VS Code and confirm the license activation.

Part B: Create a New ESP32 Project in VS Code

1. Open the PlatformIO sidebar (ant icon on the left).
2. Click on New Project.
3. Fill in the details:
 - Name: ESP32-FirstProject
 - Board: Espressif ESP32 Dev Module
 - Framework: Arduino
 - Choose your desired location → Click Finish.

4. Wait until PlatformIO finishes initializing the project.

Part C: Add Wokwi Simulation Support

After the project is created, you need to add two important files for Wokwi:

1. In the root of your project, create a new file named wokwi.toml and paste the following:

```
[wokwi] version = 1 firmware = ".pio/build/esp32dev/firmware.bin" elf = ".pio/build/esp32dev/firmware.elf"
```
2. Create a diagram.json file in the root to define hardware.
3. Right click diagram.json file and select open with, then select Text Editor.
4. Copy and paste here, the code from digram.json file in Wokwi site.

Part D: Add the code for ESP32

1. Open the file:
 - a. src/main.cpp
2. Remove default codes except #include <Arduino.h>
3. Copy and paste here, the code from sketch.ino file in Wokwi site.
4. Save the file.

Part E: Build and Run the Simulation

1. Press f1 → Click on PlatformIO:Build → (or press the ✓ icon at the bottom).
2. After successful build, press F1 → type and select: Wokwi: Start Simulator

How to Add Libraries for ESP32 in Visual Studio Code using PlatformIO

Method: Using PlatformIO Library Manager

1. In the PlatformIO sidebar, click on PIO Home → Open.
2. In the PlatformIO Home page, click on the Libraries tab (on the left menu).
3. In the search bar, type the name of the library you want to add.
 - a. Example: Type WiFiManager
4. Find the correct library in the results (for WiFiManager, choose tzapu/WiFiManager).
5. Click the Add to Project button.
6. Select your current project from the list and click Add.
7. PlatformIO will automatically add the library to your platformio.ini file.

8. After adding the library, you can use it in your code like this:

- i. `#include <WiFiManager.h>`
- ii. `WiFiManager wm;`

Lab Sheet No: 01

Title: Getting Started with ESP32 – Digital Input/Output and Wi-Fi Connectivity Basics

Aims: Introducing the fundamentals of ESP32 programming by developing and simulating digital input/output and Wi-Fi connectivity applications, including establishing Wi-Fi connections using both hardcoded credentials and the WiFiManager library, while using the Serial Monitor for debugging and real-time observation.

Tasks:

- Create and simulate ESP32 projects in the Wokwi simulator.
- Use the Serial Monitor for debugging and real-time observation.
- Connect the ESP32 to Wi-Fi using hardcoded SSID and password.
- Implement dynamic Wi-Fi configuration using the WiFiManager library.
- Combine Wi-Fi status with LED behavior for visual feedback and print connection details (IP address) only after successful connection.

ESP32



- ❖ Total Pins – 38 pins (on common 38-pin DevKit variants)
 - Ground Pins – 3 pins (GND)
 - Power Pins – 2 pins
 - 3V3 (3.3V output)
 - 5V / VIN (input voltage, usually 5V)
 - Other Pins
 - GPIO (General Purpose Input/Output) Pins
 - EN (Enable / Reset)
 - TX / RX (UART for serial communication)
 - Analog Input Pins (ADC)

Exercise 1: Connecting ESP32 to Wi-Fi

Part A: Hardcoded Wi-Fi Connection (Simple Method)

1. Create a new ESP32 project in Wokwi.
2. Write the code for hardcoded Wi-Fi connection using Wokwi-GUEST (open network, no password).
3. Run the simulation and observe the Serial Monitor (connection progress dots followed by successful connection and IP address).
4. Change the SSID to an incorrect name (e.g., "MyHomeWiFi") and observe the connection failure.

```
#include <Arduino.h>
#include <WiFi.h>

const char* ssid = "Wokwi-GUEST"; // FIXED
const char* password = "";

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\nConnecting to WiFi...");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA); // optional but good practice
  WiFi.begin(ssid, password); // FIXED

  int attempts = 0;

  while (WiFi.status() != WL_CONNECTED && attempts < 50) {
    Serial.print(".");
    delay(500); // IMPORTANT FIX
    attempts++;
  }

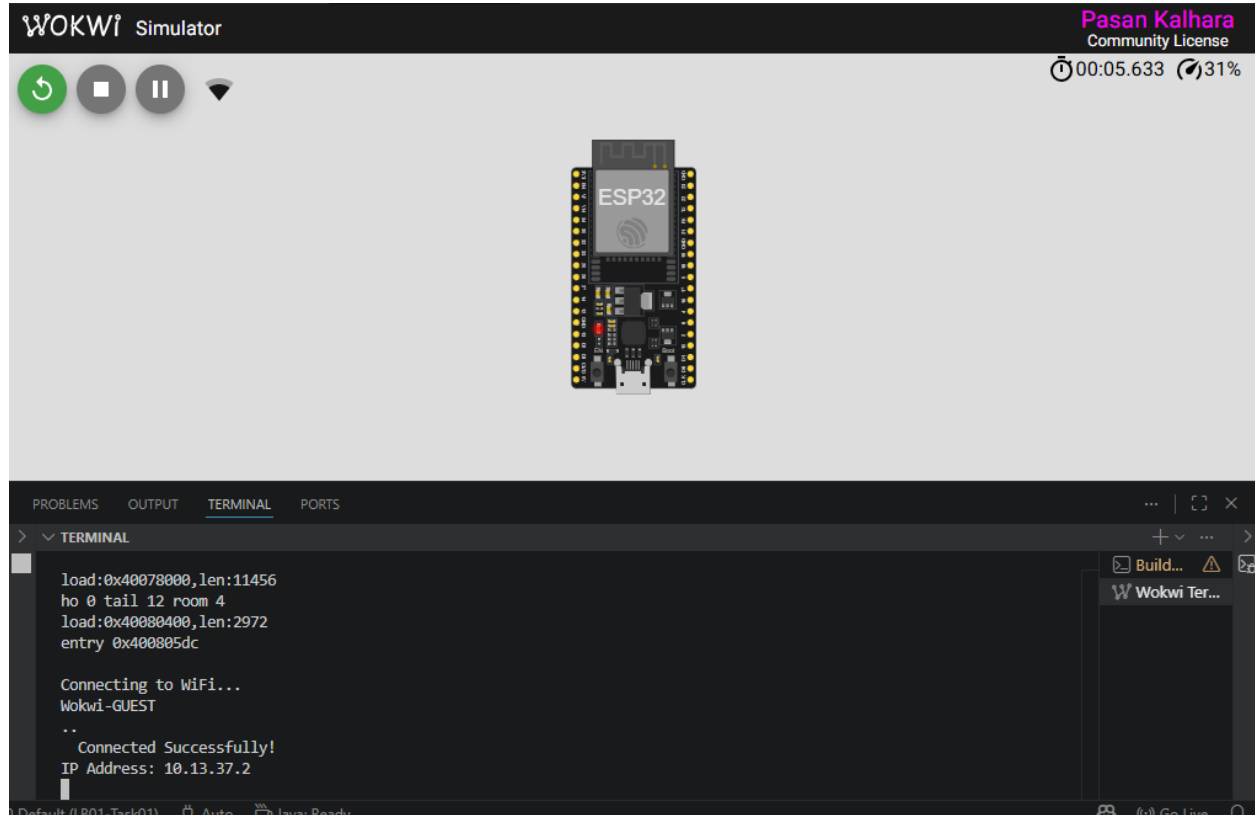
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nConnected Successfully!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
  }
}
```

```

    } else {
        Serial.println("\nFailed to connect to WiFi");
    }
}

void loop() {
}

```



Part B: Using WiFiManager (Recommended for Real Projects)

1. Add the WiFiManager library in Wokwi (via Library Manager).
2. Write and run the WiFiManager code.
3. Observe the Serial Monitor for portal creation and connection status.
4. (For real hardware) Connect your phone/laptop to the “ESP32-Config-Portal” Access Point, open a browser at 192.168.4.1, and enter your Wi-Fi SSID and password.

```

#include <WiFi.h>
#include <WiFiManager.h>
WiFiManager wm;
void setup() {

    Serial.begin(115200);
    delay(10000);
    Serial.println("\n\n ESP32 WiFi Manager Starting...");
    WiFi.mode(WIFI_STA);
    bool res = wm.autoConnect("ESP32-Config-Portal");
    if(res){
        Serial.println("Successfully connected to Wi-Fi");
        Serial.print("IP Address: ");
        Serial.println(WiFi.localIP());
    }else{
        Serial.println("\nFailed to connect.");
    }
}

void loop() {
    delay(1000);
}

```

WOKWI Simulator

Pasan Kalhara
Community License

00:15.616 29%



entry 0x400805dc

```

ESP32 WiFi Manager Starting...
*wm:AutoConnect
*wm:No wifi saved, skipping
*wm:AutoConnect: FAILED for 1 ms
*wm:StartAP with SSID: ESP32-Config-Portal
*wm:AP IP address: 192.168.4.1
*wm:Starting Web Portal

```

Default (LB01-Task02) Auto Java: Ready Go Live

Part C: Combine Wi-Fi status with LED behavior for visual feedback.

1. Blink the onboard LED slowly when not connected to Wi-Fi.
2. Blink the LED fast or keep it ON when successfully connected.
3. Print the IP address only after a successful connection.

```
#include <WiFi.h>
#include <WiFiManager.h>

WiFiManager wm;
#define LED_PIN 2 // ESP32 onboard LED (usually GPIO 2)
bool wasConnected = false;
bool ledState = false;
unsigned long previousMillis = 0;
const unsigned long DISCONNECTED_BLINK_MS = 1000;
const unsigned long CONNECTED_BLINK_MS = 200;

void blinkLed(unsigned long intervalMs) {
    if (millis() - previousMillis >= intervalMs) {
        previousMillis = millis();
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
    }
}

void setup() {
    Serial.begin(115200);
    delay(2000);
    pinMode(LED_PIN, OUTPUT);
    Serial.println("\n\nESP32 WiFi Manager Starting...");
    WiFi.mode(WIFI_STA);
    // Keep Loop running while the portal is active so LED can blink.
    wm.setConfigPortalBlocking(false);
    // Start WiFi Manager (AP portal will open if no WiFi saved)
    wm.autoConnect("ESP32-Config-Portal");
}

void loop() {
    wm.process();
    const bool isConnected = (WiFi.status() == WL_CONNECTED);
```

```

if (!isConnected) {
    // Not connected -> slow blink
    blinkLed(DISCONNECTED_BLINK_MS);
    wasConnected = false;
} else {
    // Connected -> fast blink
    blinkLed(CONNECTED_BLINK_MS);

    // Print IP only when connection is first established
    if (!wasConnected) {
        Serial.println("\nWi-Fi Connected Successfully!");
        Serial.print("IP Address: ");
        Serial.println(WiFi.localIP());
        wasConnected = true;
    }
}
}
}

```

